Análise de Riscos: Task Manager

1. Objetivo

Identificar os riscos de qualidade do sistema Task Manager, avaliar a probabilidade e o impacto de cada um e definir quais áreas devem ser testadas primeiro. Versão analisada: branch development, commit 4aaa2f6.

2. Defeitos encontrados na execução

ID	Defeito	Causa	Severidade	Risco	Situação
1	O build do backend falha porque `application.port.out` não está versionado	A regra `out/` no `.gitignore` ignorou a pasta	Alta	R6	Corrigido
2	Duas atualizações simultâneas retornam sucesso e uma delas é perdida sem aviso	`TaskEntity` sem controle de versão (`@Version`)	Média	R10	Aberto
3	A API aceita qualquer requisição sem autenticação	Sem Spring Security no projeto	Média	R5	Aberto

3. Metodologia

Cada risco recebe uma nota de 1 a 3 para a probabilidade de ocorrer e outra de 1 a 3 para o impacto caso ocorra, e a multiplicação das duas (probabilidade × impacto) define o nível do risco (baixo de 1 a 2, médio de 3 a 4 e alto de 6 a 9), que determina a prioridade dos testes.

Nota	Probabilidade	Impacto
1	Baixa: dificilmente ocorre	Baixo: problema pequeno, tem contorno fácil
2	Média: pode ocorrer	Médio: prejudica uma funcionalidade importante
3	Alta: ocorre com frequência	Alto: sistema para, perde dados ou fica inseguro

Nível de risco = Probabilidade × Impacto

- 1 e 2: baixo, testar se houver tempo
- 3 e 4: médio, testar normalmente
- 6 e 9: alto, testar com prioridade

4. Riscos identificados

ID	Risco	Área	Prob.	Imp.	Nível
R1	Validação falha e deixa passar dados incorretos (título ou descrição vazios, status inválido)	API / validação	2	2	4 (Médio)
R2	Perda ou corrupção de dados: a task é salva, mas a falha posterior (evento) não é tratada junto, sem transação que una as duas operações	API / PostgreSQL	2	3	6 (Alto)
R3	Falha nas migrações Liquibase impede o sistema de subir ou deixa o banco inconsistente	Banco de dados	2	3	6 (Alto)
R4	Eventos Kafka perdidos ou inválidos: a exceção do envio é capturada e só registrada em log, o	Mensageria	3	2	6 (Alto)

	resultado do `send` não é verificado, o JSON é montado à mão e o `escape` ignora quebras de linha e caracteres de controle				
R5	API sem autenticação ou autorização: qualquer pessoa lê, altera e apaga tasks	Segurança	3	2	6 (Alto)
R6	Ambiente não sobe ou não compila (Docker, Kafka, `.env`, arquivos faltando no repositório)	Infraestrutura / build	3	2	6 (Alto)
R7	Operações com ID inexistente ou dados malformados retornam resposta ou status HTTP incorretos	API / tratamento de erros	2	2	4 (Médio)
R8	Frontend não exibe corretamente erros por campo, de rede ou de timeout	Interface	2	2	4 (Médio)
R9	Credenciais padrão fracas em `infra/.env.example` usadas sem troca	Configuração / segurança	3	1	3 (Médio)
R10	Atualizações simultâneas sobrescrevem dados: `update` busca e salva sem controle de versão	API / concorrência	2	2	4 (Médio)

5. Matriz de probabilidade x impacto

Probabilidade \ Impacto	Baixo (1)	Médio (2)	Alto (3)

Alta (3)	R9 (3)	R4, R5, R6 (6)	—
Média (2)	—	R1, R7, R8, R10 (4)	R2, R3 (6)
Baixa (1)	—	—	—

6. Prioridades de teste

Prioridade alta (R2, R3, R4, R5, R6):

- Testes de integração do CRUD completo, conferindo no banco se o dado foi gravado (R2).
- Subir o banco vazio e verificar as migrações Liquibase (R3).
- Derrubar o Kafka (`docker compose stop kafka`), criar uma task e confirmar que a API responde sucesso sem evento publicado (R4).
- Criar task com quebra de linha e tabulação na descrição e verificar se o payload no tópico é um JSON válido (R4).
- Chamar a API sem credenciais e documentar o resultado (R5).
- Validar o build a partir de um clone limpo, no CI e localmente (R6).

Prioridade média (R1, R7, R8, R9, R10):

- Valor limite e classes de equivalência na validação, incluindo o cenário do README (`title` e `description` vazios, esperando HTTP 400 com `fieldErrors`) (R1).
- GET, PUT e DELETE com ID inexistente, esperando 404 (R7).
- Testes de componente no frontend e inspeção manual da interface (R8).
- Inspeção do `.env.example` (R9).
- Duas atualizações simultâneas da mesma task (R10).

Prioridade baixa:

Na análise inicial nenhum risco ficou em prioridade baixa.

7. Reavaliação dos riscos

As notas de probabilidade e impacto deste documento representam a avaliação inicial, feita a partir da leitura do código e da execução do sistema. Elas serão revisadas depois da execução dos testes: riscos confirmados por defeitos reais poderão subir de nível, e riscos que não se materializarem poderão descer. Assim, a análise é um processo contínuo, e não um documento fixo.

8. Conclusão

Os maiores riscos do Task Manager estão em persistência de dados, migrações do banco, integração com Kafka, ausência de controle de acesso e reprodutibilidade do build. Três deles já têm evidência concreta: o defeito D1 (R6), a ausência de autenticação (R5) e as falhas silenciosas na publicação de eventos (R4), esta identificada na leitura do código. Esses pontos receberão mais casos de teste e serão automatizados primeiro. Validação, tratamento de erros, interface, credenciais padrão e concorrência vêm em seguida.